

Model Monitor Bias Report: AI-Powered Apple Leaf Specialist

- **Name:** Aktham Almomani
- **Group:** 7

Introduction

This notebook operationalizes post-training bias analysis for the Apple Leaf classifier using **Amazon SageMaker Clarify**. It deploys the trained PyTorch model with request/response capture, builds an evaluation table with predictions and a simple “brightness” facet, and generates a Clarify bias report.

We'll cover the following item:

- Model deployment with `DataCaptureConfig`.
- Request generation and capture verification in S3.
- Local batch inference to create `eval.csv`.
- Clarify configuration: `DataConfig`, `BiasConfig`, `ModelPredictedLabelConfig`.
- Executing `run_post_training_bias` and retrieving the report artifacts.
- Reading key metrics and known caveats (e.g., CDDPL requires group variable).

Setup and Deployment

Let's start by Initializing SageMaker session and role, reference the trained model in S3, and deploy a `PyTorchModel` to a named endpoint. Enable data capture at 10% to a dedicated S3 prefix for later auditing and monitoring.

```
from sagemaker.model_monitor import DataCaptureConfig
from sagemaker.pytorch import PyTorchModel
import sagemaker, boto3
from pathlib import Path

sess = sagemaker.Session()
role = sagemaker.get_execution_role()
bucket = sess.default_bucket()

# Trained artifact in S3:
model_data =
"s3://sagemaker-us-east-1-533266958221/apple/outputs/pytorch-training-
2025-10-16-04-42-30-265/output/model.tar.gz"

pyt_model = PyTorchModel(
    model_data=model_data,
    role=role,
    framework_version="2.3",
```

```

    py_version="py311",
    entry_point="inference.py",
    source_dir="src",
    sagemaker_session=sess,
)

rt_endpoint_name = "apple-rt-with-capture-v2"
capture_s3_uri = f"s3://{bucket}/sagemaker/{rt_endpoint_name}/data-capture"

capture_cfg = DataCaptureConfig(
    enable_capture=True,
    sampling_percentage=10, # 10% of traffic
    destination_s3_uri=capture_s3_uri,
    capture_options=["REQUEST", "RESPONSE"],
    #capture_content_type_header={"JsonContentTypes":
["application/json"]},
)

predictor_rt = pyt_model.deploy(
    endpoint_name=rt_endpoint_name,
    initial_instance_count=1,
    instance_type="ml.m5.large",
    data_capture_config=capture_cfg,
)

print("Deployed:", rt_endpoint_name)
print("Capture to:", capture_s3_uri)

sagemaker.config INFO - Not applying SDK defaults from location:
/etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location:
/home/sagemaker-user/.config/sagemaker/config.yaml
-----!Deployed: apple-rt-with-capture-v2
Capture to: s3://sagemaker-us-east-1-533266958221/sagemaker/apple-rt-with-capture-v2/data-capture

```

Warm-Up and Data Capture

Here let's send ~20 JSON requests (base64 images) through the predictor to generate traffic. List the capture prefix to confirm JSON Lines files were written under the endpoint/date/hour hierarchy in S3.

```

import base64, json, zipfile, random

# reuse the predictor just deployed:
predictor_rt.serializer = sagemaker.serializers.JSONSerializer()
predictor_rt.deserializer = sagemaker.deserializers.JSONDeserializer()

```

```

# Here let's pick ~20 test images and invoke:
local_zip = Path.home() / "AI_Powered_Apple_Leaf_Specialist" /
"test.zip"
extract_dir = Path("/tmp/capture_warmup");
extract_dir.mkdir(parents=True, exist_ok=True)
with zipfile.ZipFile(local_zip) as z: z.extractall(extract_dir)

imgs = list(extract_dir.rglob("*.jpg"))
batch = random.sample(imgs, k=min(20, len(imgs)))

for p in batch:
    with open(p, "rb") as f:
        payload = {"b64": base64.b64encode(f.read()).decode()}
        _ = predictor_rt.predict(payload)

print("Sent", len(batch), "requests")

Sent 20 requests

import datetime as dt

s3 = boto3.client("s3")
bucket = sess.default_bucket()
prefix = f"sagemaker/{predictor_rt.endpoint_name}/data-capture/"

resp = s3.list_objects_v2(Bucket=bucket, Prefix=prefix)
count = sum(1 for _ in resp.get("Contents", []))
print("Objects now under", f"s3://{bucket}/{prefix}", "->", count)
for o in resp.get("Contents", [])[:5]: # peek a few
    print(o["Key"], o["Size"])

Objects now under
s3://sagemaker-us-east-1-533266958221/sagemaker/apple-rt-with-capture-
v2/data-capture/ -> 1
sagemaker/apple-rt-with-capture-v2/data-capture/apple-rt-with-capture-
v2/AllTraffic/2025/10/17/03/52-35-945-42e7ceae-d11e-4913-84e0-
86f76b5cd0d8.jsonl 144443

```

Build Evaluation Dataset

Now let's create an ImageFolder-based loader from the test split, run local inference with the trained weights, and save a flat table with true label, predicted label, per-class probabilities, and a derived `brightness_bin` facet (bright/dim). Persist to `eval.csv`.

```

import io, numpy as np
from PIL import Image
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import pandas as pd

```

```

# Our project paths:
studio_home = Path.home() / "AI_Powered_Apple_Leaf_Specialist"
test_zip     = studio_home / "test.zip"
work         = Path("/tmp/clarify_eval"); work.mkdir(exist_ok=True)

# our splits paths:
def pick_split_root(base: Path) -> Path:
    for name in ["test", "val", "validation"]:
        p = base / name
        if p.exists() and any(x.is_dir() for x in p.iterdir()):
            return p
    if base.exists() and any(x.is_dir() for x in base.iterdir()):
        return base
    raise FileNotFoundError(f"No class folders under {base} (or {base}/test).")

ds_root = pick_split_root(test_root)

# Build dataset / loader:
IMG_SIZE = 256
tfm = transforms.Compose([transforms.Resize((IMG_SIZE, IMG_SIZE)),
                           transforms.ToTensor()])
test_ds = datasets.ImageFolder(ds_root, transform=tfm)
LABELS = test_ds.classes
test_dl = DataLoader(test_ds, batch_size=32, shuffle=False,
                     num_workers=2)

# load trained weights:
model_path = studio_home / "model_artifacts_local" / "model.pt"
model = models.resnet18(weights=None)
model.fc = nn.Linear(model.fc.in_features, len(LABELS))
state = torch.load(model_path, map_location="cpu")
model.load_state_dict(state); model.eval()

# helper: brightness facet (mean intensity after resize):
def brightness_bin(pil_rgb, thr=0.5):
    x = np.asarray(pil_rgb.convert("L"), dtype=np.float32)/255.0
    return "bright" if x.mean() >= thr else "dim"

rows = []
inv_tf = transforms.ToPILImage()
with torch.inference_mode():
    for x, y in test_dl:
        logits = model(x); probs = torch.softmax(logits,
dim=1).cpu().numpy()
        pred = probs.argmax(1)
        for i in range(x.size(0)):
            true_lbl = LABELS[int(y[i].item())]
            pred_lbl = LABELS[int(pred[i])]

```

```

model          # approximate brightness on the resized image we fed the
                facet      = brightness_bin(inv_tf(x[i]))
                rows.append(
                    {"label": true_lbl, "pred_label": pred_lbl,
                     **{f"prob_{LABELS[j]}": float(probs[i,j]) for j in
range(len(LABELS))},
                     "brightness_bin": facet}
                )

```

```

eval_df = pd.DataFrame(rows)
eval_csv = work / "eval.csv"
eval_df.to_csv(eval_csv, index=False)
eval_df.head()

```

	label	pred_label	prob_black_rot	prob_healthy	prob_rust
prob_scab \					
0	black_rot	black_rot	0.999207	2.549132e-05	0.000155
0.000612					
1	black_rot	black_rot	0.994976	2.735349e-10	0.005022
0.000002					
2	black_rot	black_rot	0.999674	8.571298e-06	0.000242
0.000076					
3	black_rot	black_rot	0.997890	2.089768e-06	0.001752
0.000356					
4	black_rot	black_rot	0.999209	2.500356e-07	0.000713
0.000078					

	brightness_bin
0	dim
1	bright
2	dim
3	bright
4	dim

Configure Clarify Bias Analysis

Here let's upload `eval.csv` to S3 and define:

- **DataConfig:** input path, output path, headers, label columns.
- **BiasConfig:** facet `brightness_bin`; "positive" labels = disease classes.
- **ModelPredictedLabelConfig:** use `pred_label` from the table.
- **Run Post-Training Bias Metrics:** Launch a Clarify processing job on a small instance to compute metrics such as AD, DAR, DI, DPPL, RD, and more.
- **Reports and Artifacts:** Clarify writes an HTML and PDF report to the configured S3 output prefix along with JSON summaries. The report visualizes global and facet-level metrics and includes links to open the analysis directly.

```

from sagemaker.clarify import (
    SageMakerClarifyProcessor, DataConfig,
    BiasConfig, ModelPredictedLabelConfig
)
import time

sess = sagemaker.Session()
role = sagemaker.get_execution_role()
bucket = sess.default_bucket()

def noslash(uri: str) -> str:
    return uri.rstrip("/")

stamp = time.strftime("%Y%m%d-%H%M%S")
input_s3 = noslash(f"s3://{bucket}/apple/clarify/input")
output_s3 = noslash(f"s3://{bucket}/apple/clarify/reports/{stamp}")

# Upload eval_csv:
sess.upload_data(str(eval_csv), bucket=bucket,
key_prefix="apple/clarify/input")

data_cfg = DataConfig(
    s3_data_input_path=input_s3,
    s3_output_path=output_s3,
    label="label",
    predicted_label="pred_label",
    headers=list(eval_df.columns),
    dataset_type="text/csv",
)

# Facet and positive outcome:
pos_is_disease = ["black_rot", "rust", "scab"]
bias_cfg = BiasConfig(
    label_values_or_threshold=pos_is_disease,
    facet_name="brightness_bin",
)

pred_cfg = ModelPredictedLabelConfig(label="pred_label")

clarify = SageMakerClarifyProcessor(
    role=role,
    instance_count=1,
    instance_type="ml.m5.large",
    sagemaker_session=sess,
)

clarify.run_post_training_bias(
    data_config=data_cfg,
    data_bias_config=bias_cfg,
    model_predicted_label_config=pred_cfg
)

```

```
print("Bias report prefix:", output_s3)
print("Open:", output_s3 + "/analysis.html")
```

```
INFO:sagemaker.image_uris:Ignoring unnecessary instance type: None.
INFO:sagemaker.clarify:Analysis Config: {'dataset_type': 'text/csv',
'headers': ['label', 'pred_label', 'prob_black_rot', 'prob_healthy',
'prob_rust', 'prob_scab', 'brightness_bin'], 'label': 'label',
'predicted_label': 'pred_label', 'label_values_or_threshold':
['black_rot', 'rust', 'scab'], 'facet': [{'name_or_index':
'brightness_bin'}], 'methods': {'report': {'name': 'report', 'title':
'Analysis Report'}, 'post_training_bias': {'methods': 'all'}}}
INFO:sagemaker:Creating processing-job with name Clarify-Posttraining-
Bias-2025-10-17-04-38-46-113
```

```
.....sagemaker.config INFO - Not applying SDK
defaults from location: /etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location:
/root/.config/sagemaker/config.yaml
We are not in a supported iso region, /bin/sh exiting gracefully with
no changes.
```

```
WARNING:root:logging.conf not found when configuring logging, using
default logging configuration.
```

```
INFO:sagemaker-clarify-processing:Starting SageMaker Clarify
Processing job
```

```
INFO:analyzer.data_loading.data_loader_util:Analysis config path:
/opt/ml/processing/input/config/analysis_config.json
```

```
INFO:analyzer.data_loading.data_loader_util:Analysis result path:
/opt/ml/processing/output
```

```
INFO:analyzer.data_loading.data_loader_util:This host is algo-1.
```

```
INFO:analyzer.data_loading.data_loader_util:This host is the leader.
```

```
INFO:analyzer.data_loading.data_loader_util:Number of hosts in the
cluster is 1.
```

```
INFO:sagemaker-clarify-processing:Running Python / Pandas based
analyzer.
```

```
INFO:analyzer.data_loading.data_loader_factory:Dataset type: text/csv
uri: /opt/ml/processing/input/data
```

```
INFO:sagemaker-clarify-processing:Loading dataset...
```

```
INFO:sagemaker-clarify-processing:Loaded dataset. Dataset info:
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 475 entries, 0 to 474
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	prob_black_rot	475 non-null	float64
1	prob_healthy	475 non-null	float64
2	prob_rust	475 non-null	float64
3	prob_scab	475 non-null	float64
4	brightness_bin	475 non-null	object

```
dtypes: float64(4), object(1)
```

```

memory usage: 18.7+ KB
INFO:sagemaker-clarify-
processing:=====
INFO:sagemaker-clarify-processing:Calculating post-training bias
metrics
INFO:sagemaker-clarify-
processing:=====
INFO:smclarify.bias.metrics.common:Column label with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column brightness_bin with data
uniqueness fraction 0.004210526315789474 is classified as a CATEGORICAL
column
INFO:smclarify.bias.metrics.common:Column label with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column None with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
ERROR:smclarify.bias.report:CDDPL metrics failed
Traceback (most recent call last):
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/report.py",
line 322, in _categorical_metric_call_wrapper
    metric_value = smclarify.bias.metrics.call_metric(
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/__init_
_.py", line 27, in call_metric
    return metric(**{key: kwargs[key] for key in
inspect.signature(metric).parameters.keys()})
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/posttra
ining.py", line 276, in CDDPL
    return common.CDD(feature, sensitive_facet_index,
positive_predicted_label_index, group_variable)
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/common.
py", line 99, in CDD
    raise ValueError("Group variable is empty or not provided")
ValueError: Group variable is empty or not provided
ERROR:smclarify.bias.report:CDDPL metrics failed
Traceback (most recent call last):
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/report.py",
line 322, in _categorical_metric_call_wrapper
    metric_value = smclarify.bias.metrics.call_metric(
  File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/__init_
_.py", line 27, in call_metric
    return metric(**{key: kwargs[key] for key in
inspect.signature(metric).parameters.keys()})
  File

```



```

"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/posttra
ining.py", line 276, in CDDPL
    return common.CDD(feature, sensitive_facet_index,
positive_predicted_label_index, group_variable)
File
"/usr/local/lib/python3.9/site-packages/smclarify/bias/metrics/common.
py", line 99, in CDD
    raise ValueError("Group variable is empty or not provided")
ValueError: Group variable is empty or not provided
INFO:sagemaker-clarify-processing:Calculated global analysis with
predictor
INFO:sagemaker-clarify-
processing:=====
INFO:sagemaker-clarify-processing:Calculating bias statistics for
report
INFO:sagemaker-clarify-
processing:=====
INFO:smclarify.bias.metrics.common:Column label with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column brightness_bin with data
uniqueness fraction 0.004210526315789474 is classified as a CATEGORICAL
column
INFO:smclarify.bias.metrics.common:Column label with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column None with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column label with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:smclarify.bias.metrics.common:Column None with data uniqueness
fraction 0.008421052631578947 is classified as a CATEGORICAL column
INFO:analyzer.utils.spark_util:Converting Pandas DataFrame to
SparkDataFrame for computing report metadata
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
25/10/17 04:43:17 WARN NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where
applicable
#015[Stage 0:>
(0 + 2) / 2]#015#015
#015INFO:sagemaker-clarify-processing:Calculated global analysis
without predictor
INFO:sagemaker-clarify-processing:Collected analyses:
{'version': '1.0', 'post_training_bias_metrics': {'label': 'label',
'facets': {'brightness_bin': [{'value_or_threshold': 'dim', 'metrics':
[{'name': 'AD', 'description': 'Accuracy Difference (AD)', 'value':
0.0078125}, {'name': 'CDDPL', 'description': 'Conditional Demographic
Disparity in Predicted Labels (CDDPL)', 'value': None, 'error': 'Group
variable is empty or not provided'}, {'name': 'DAR', 'description':

```

```

'Difference in Acceptance Rates (DAR)', 'value': 0.0}, {'name': 'DCA',
'description': 'Difference in Conditional Acceptance (DCA)', 'value':
-0.015463917525773141}, {'name': 'DCR', 'description': 'Difference in
Conditional Rejection (DCR)', 'value': -0.015789473684210575},
{'name': 'DI', 'description': 'Disparate Impact (DI)', 'value':
1.4366861979166667}, {'name': 'DPPL', 'description': 'Difference in
Positive Proportions in Predicted Labels (DPPL)', 'value': -
0.1535599816849817}, {'name': 'DRR', 'description': 'Difference in
Rejection Rates (DRR)', 'value': -0.015789473684210575}, {'name':
'FT', 'description': 'Flip Test (FT)', 'value': 0.0}, {'name': 'GE',
'description': 'Generalized Entropy (GE)', 'value':
0.00317796610169479}, {'name': 'RD', 'description': 'Recall Difference
(RD)', 'value': 0.015228426395939132}, {'name': 'SD', 'description':
'Specificity Difference (SD)', 'value': 0.0}, {'name': 'TE',
'description': 'Treatment Equality (TE)', 'value': 'Infinity'}}],
{'value_or_threshold': 'bright', 'metrics': [{'name': 'AD',
'description': 'Accuracy Difference (AD)', 'value': -0.0078125},
{'name': 'CDDPL', 'description': 'Conditional Demographic Disparity in
Predicted Labels (CDDPL)', 'value': None, 'error': 'Group variable is
empty or not provided'}, {'name': 'DAR', 'description': 'Difference in
Acceptance Rates (DAR)', 'value': 0.0}, {'name': 'DCA', 'description':
'Difference in Conditional Acceptance (DCA)', 'value':
0.015463917525773141}, {'name': 'DCR', 'description': 'Difference in
Conditional Rejection (DCR)', 'value': 0.015789473684210575}, {'name':
'DI', 'description': 'Disparate Impact (DI)', 'value':
0.6960462218194177}, {'name': 'DPPL', 'description': 'Difference in
Positive Proportions in Predicted Labels (DPPL)', 'value':
0.1535599816849817}, {'name': 'DRR', 'description': 'Difference in
Rejection Rates (DRR)', 'value': 0.015789473684210575}, {'name': 'FT',
'description': 'Flip Test (FT)', 'value': 0.0}, {'name': 'GE',
'description': 'Generalized Entropy (GE)', 'value':
0.00317796610169479}, {'name': 'RD', 'description': 'Recall Difference
(RD)', 'value': -0.015228426395939132}, {'name': 'SD', 'description':
'Specificity Difference (SD)', 'value': 0.0}, {'name': 'TE',
'description': 'Treatment Equality (TE)', 'value': '-Infinity'}}]]],
'label_value_or_threshold': 'black_rot,rust,scab'}}
INFO:analyzer.utils.util:['jupyter', 'nbconvert', '--to', 'html', '--
output', '/opt/ml/processing/output/report.html',
'/opt/ml/processing/output/report.ipynb', '--template', 'sagemaker-
xai']
[NbConvertApp] Converting notebook
/opt/ml/processing/output/report.ipynb to html
[NbConvertApp] Writing 617397 bytes to
/opt/ml/processing/output/report.html
INFO:analyzer.utils.util:['wkhtmltopdf', '-q', '--enable-local-file-
access', '/opt/ml/processing/output/report.html',
'/opt/ml/processing/output/report.pdf']
INFO:analyzer.utils.system_util:exit_message: Completed: SageMaker XAI
Analyzer ran successfully

```

INFO:py4j.clientserver:Closing down clientserver connection

Bias report prefix:

s3://sagemaker-us-east-1-533266958221/apple/clarify/reports/20251017-043846

Open:

s3://sagemaker-us-east-1-533266958221/apple/clarify/reports/20251017-043846/analysis.html